

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота № 8

з дисципліни «Алгоритмізація та програмування»

на тему: «Реалізація алгоритмів сортування та робота з файлами на мові C ++»

XAI.301. G3.319a.01 ПР

Виконала: здобувачка 1 курсу, групи 319а
напряму підготовки (спеціальності)

G3 “Електрична Інженерія”

Вікторія БАБІНА

(ім'я і прізвище)

Прийняв: асис.,

_____ Євгеній ПЯВКА

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_n.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.№6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_n.txt». Array67. Дан цілочисельний масив розміру N. Збільшити всі непарні числа, що містяться в масиві, на початкове значення останнього непарного числа. Якщо непарні числа в масиві відсутні, то залишити масив без змін., ім'я файлу і необхідні змінні ввести з консолі.

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_n.txt» з елементами вихідного двовимірного масиву (n - номер варіанта). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл. Matrix19. Дана матриця розміру $M \times N$. Для кожного рядка матриці знайти суму її елементів., ім'я файлу і необхідні змінні ввести з консолі.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

3	Вибір	Зростання	Цілий
---	-------	-----------	-------

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами. Структурувати проєкт програми для виконання завдань 1-3 наступним чином:

```
main.cpp //основна функція і три функції для 3х завдань
array_utils.h //заголовки функцій для роботи з
//одновимірними масивами (завдання1,3)
array_utils.cpp //визначення функцій для роботи з
//одновимірними масивами (завдання1,3)
matrix_utils.h //заголовки функцій для роботи з
//двовимірними масивами (завдання2)
matrix_utils.cpp //визначення функцій для роботи з
//двовимірними масивами (завдання2)
```

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Aggra67. Дан цілочисельний масив розміру N . Необхідно збільшити всі непарні числа масиву на значення останнього непарного числа. Якщо непарні числа відсутні — масив не змінюється.

Вхідні дані (ім'я, опис, тип, обмеження):

Filename-ім'я вхідного файлу, string, довільний

n -кількість елементів, int, $2 \leq n \leq 20$

arr-масив чисел, int, -1000 ... 1000

Вихідні дані (ім'я, опис, тип):

Arr-змінений масив, int

filename_out-ім'я вихідного файлу, string

Алгоритм вирішення показано нижче

1. Відкрити файл для читання.
2. Зчитати кількість елементів n .
3. Зчитати елементи масиву.
4. Знайти останнє непарне число.
5. Якщо таке число існує:
 - пройти масив ще раз
 - додати це число до кожного непарного елемента
6. Відкрити файл для запису.

7. Записати результат у файл.

Алгоритм зчитування з файлу

1. Відкрити файл у режимі читання.
2. Зчитати кількість елементів n .
3. Зчитати n елементів масиву.
4. Закрити файл.

Алгоритм запису у файл

1. Відкрити файл у режимі запису.
2. Записати кількість елементів n .
3. Записати всі елементи масиву.
4. Закрити файл.

Лістинг коду вирішення задачі наведено в дод. А (стор. 8).

Екран роботи програми показаний на рис. Б.1.

Завдання 2.

Matrix19. Дана матриця розміру $M \times N$. Для кожного рядка знайти суму її елементів.

Вхідні дані

Filename-ім'я файлу, string, довільний

m -кількість рядків, int, $2 \leq m \leq 20$

n -кількість стовпців, int, $2 \leq n \leq 20$

matrix-матриця, int, -1000 ... 1000

Вихідні дані

Sum-суми елементів рядків, int

Алгоритм розв'язання

1. Відкрити файл.
2. Зчитати значення m і n .
3. Зчитати елементи матриці.
4. Для кожного рядка обчислити суму елементів.
5. Відкрити файл у режимі дописування.
6. Записати суми у файл.
7. Закрити файл.

Алгоритм зчитування матриці

1. Відкрити файл у режимі читання.
2. Зчитати m і n .
3. Зчитати елементи матриці по рядках.
4. Закрити файл.

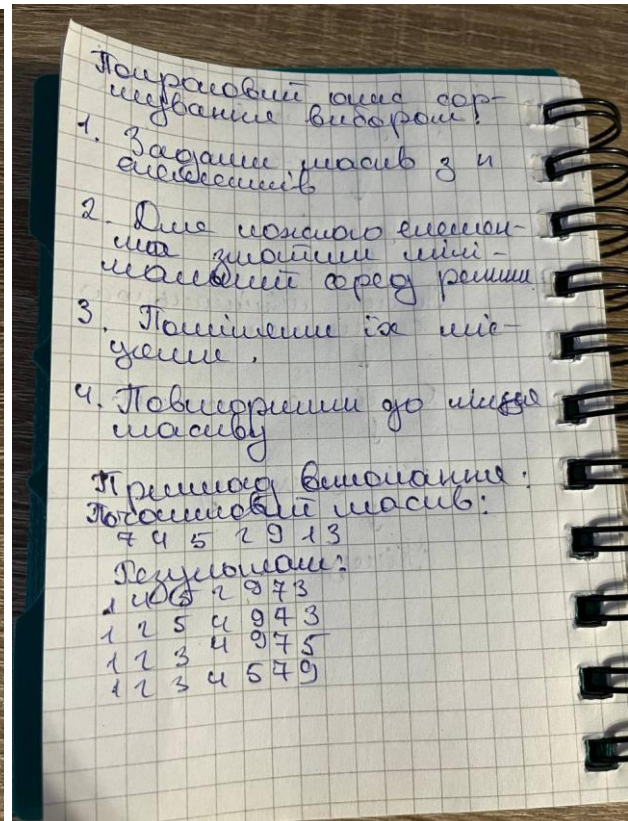
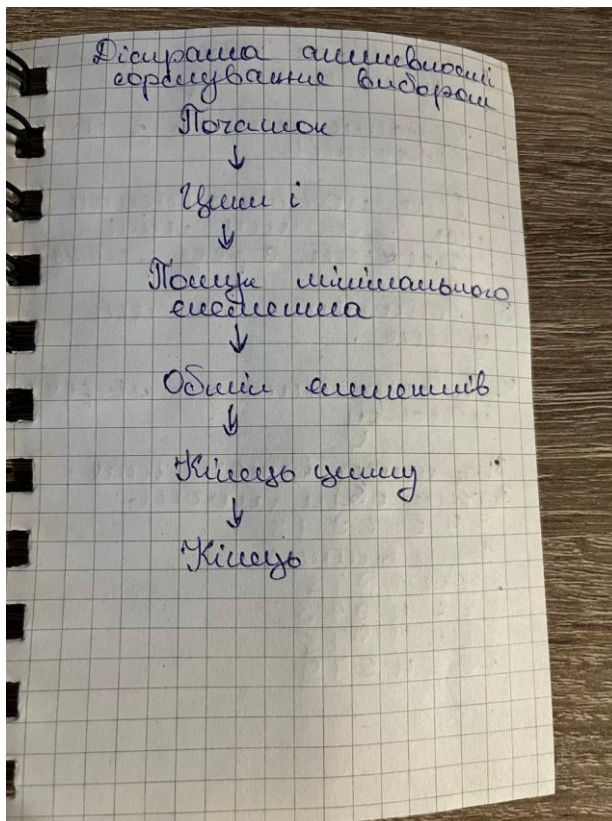
Алгоритм запису результатів

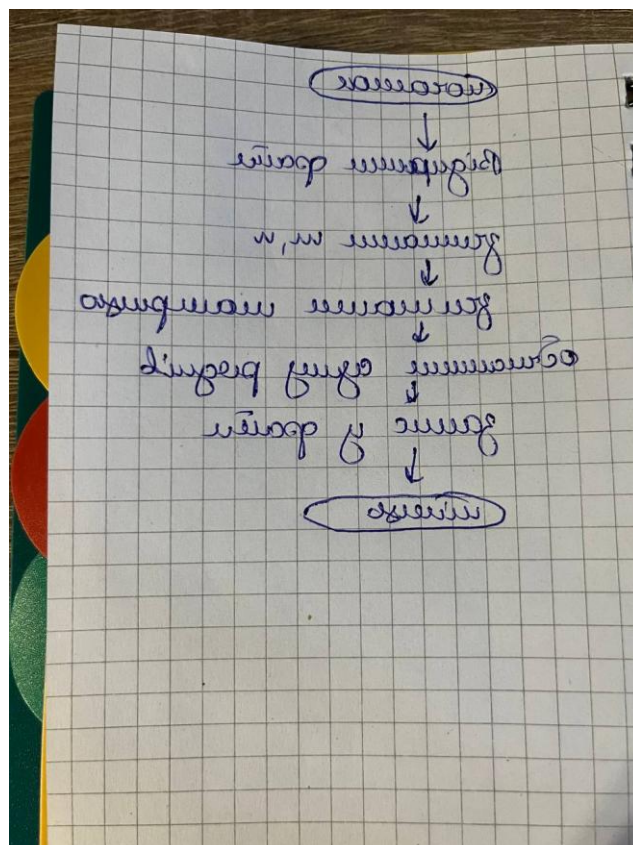
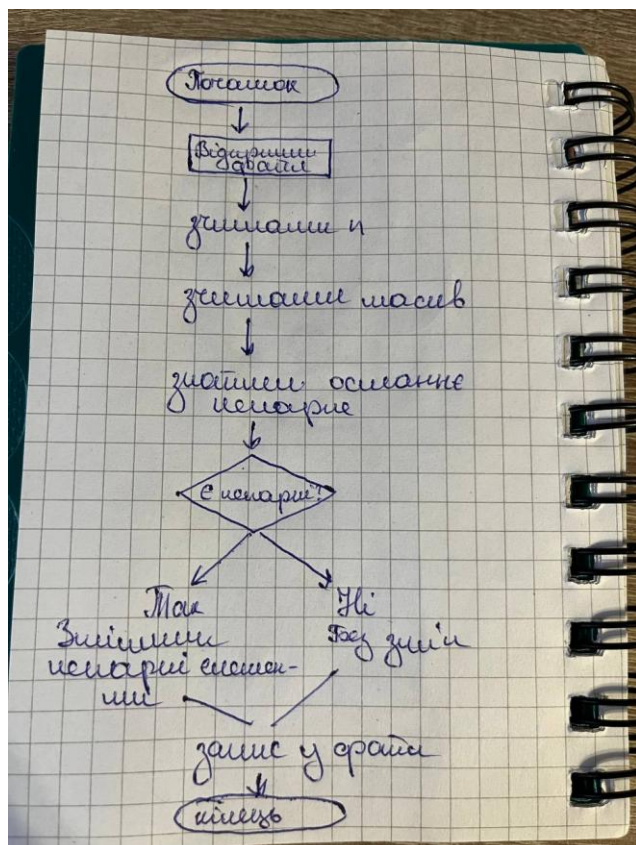
1. Відкрити файл у режимі дописування.
2. Обчислити суму елементів кожного рядка.
3. Записати отримані суми у файл.
4. Закрити файл.

Лістинг коду вирішення задачі наведено в дод. А (стор. 8).

Екран роботи програми показаний на рис. Б.2.

Завдання 3.





Завдання 5.

1. b
2. b
3. c
4. c
5. c
6. Спочатку ми проходимо масив перший раз, щоб знайти останнє непарне число і запам'ятати його. Потім ми проходимо масив вдруге, щоб замінити всі інші непарні числа на це збережене значення. За один прохід не вийде, тому що коли ми йдемо з початку, ми ще не знаємо, яке число буде останнім непарним.
7. Треба змінити умову в циклах. Скрізь, де ми перевіряємо $\text{if}(\text{число} \% 2 \neq 0)$ (непарне), потрібно написати $\text{if}(\text{число} \% 2 == 0)$ (парне). Логіка пошуку останнього елемента (тепер парного) і заміни всіх інших лишиться такою самою.
8. Помилка: Забути закрити файл (`file.close()`). Через це дані можуть не зберегтися. Як уникнути: Завжди викликати `file.close()` після завершення роботи з файлом.

Помилка: Не перевірити, чи відкрився файл. Якщо файл (наприклад, для читання) не існує, програма може вилетіти. Як уникнути: Додати перевірку: `if (!file) { cout << "Помилка відкриття файлу"; return; }`

9. Вкладений цикл потрібен, щоб пройти по всіх елементах таблиці (матриці).

Зовнішній цикл (наприклад, `for (int i = 0; i < rows; i++)`) відповідає за перехід по рядках.

Внутрішній цикл (`for (int j = 0; j < cols; j++)`) всередині кожного рядка перебирає всі стовпчики (елементи цього рядка).

10. Можна зробити універсальними функції для:

- ведення масиву з клавіатури.
 - Виведення масиву на екран.
 - Заповнення масиву випадковими числами.
 - Алгоритм пошуку останнього непарного елемента.
- Їх можна винести в окремий файл (наприклад, `array_utils.cpp`) і потім підключати до різних задач, щоб не писати той самий код багато разів.

ВИСНОВКИ

У ході виконання лабораторної роботи було вивчено методи обробки одновимірних та двовимірних масивів мовою C++. Закріплено навички роботи з файлами, зокрема зчитування та запису даних. Було реалізовано алгоритми обробки масивів та матриць, а також отримано практичні навички структуризації програмного коду.

ДОДАТОК А

Лістинг коду програми

```

#include <iostream>
#include "array_utils.h"
#include "matrix_utils.h"

using namespace std;

int main()
{
    string array_in, array_out;
    int arr[N], n;

    cout << "Input array file: ";
    cin >> array_in;

    cout << "Output array file: ";
    cin >> array_out;

    get_mas(array_in, arr, n);
    transform_array(arr, n);
    show_mas(array_out, arr, n);

    string matrix_file;
    int matrix[M][N];
    int m;

    cout << "Input matrix file: ";
    cin >> matrix_file;

    get_matrix(matrix_file, matrix, m, n);
    write_row_sums(matrix_file, matrix, m, n);

    cout << "Done!" << endl;

    return 0;
}

#ifndef MATRIX_UTILS_H
#define MATRIX_UTILS_H

#include <string>

const int M = 20;
const int N = 20;

void get_matrix(std::string filename, int matrix[M][N], int &m, int &n);
void write_row_sums(std::string filename, int matrix[M][N], int m, int n);

#endif

#include <fstream>
#include "matrix_utils.h"

using namespace std;

void get_matrix(string filename, int matrix[M][N], int &m, int &n)
{
    ifstream file(filename);

    file >> m >> n;

    for (int i = 0; i < m; i++)

```

```

        for (int j = 0; j < n; j++)
            file >> matrix[i][j];

    file.close();
}

void write_row_sums(string filename, int matrix[M][N], int m, int n)
{
    ofstream file(filename, ios::app);

    file << "\\nRow sums:\\n";

    for (int i = 0; i < m; i++)
    {
        int sum = 0;

        for (int j = 0; j < n; j++)
            sum += matrix[i][j];

        file << sum << endl;
    }

    file.close();
}

#include <fstream>
#include "array_utils.h"

using namespace std;

void get_mas(string filename, int arr[N], int &n)
{
    ifstream file(filename);

    file >> n;

    for (int i = 0; i < n; i++)
        file >> arr[i];

    file.close();
}

void show_mas(string filename, int arr[N], int n)
{
    ofstream file(filename);

    file << n << endl;

    for (int i = 0; i < n; i++)
        file << arr[i] << " ";

    file.close();
}

void transform_array(int arr[N], int n)
{
    int lastOdd = 0;
    bool found = false;

    for (int i = 0; i < n; i++)
    {
        if (arr[i] % 2 != 0)
        {
            lastOdd = arr[i];
            found = true;
        }
    }
}

```

```

    }
}

if (found)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] % 2 != 0)
            arr[i] += lastOdd;
    }
}

#endif

#ifdef ARRAY_UTILS_H

#include <string>

const int N = 20;

void get_mas(std::string filename, int arr[N], int &n);
void show_mas(std::string filename, int arr[N], int n);
void transform_array(int arr[N], int n);

#endif

```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

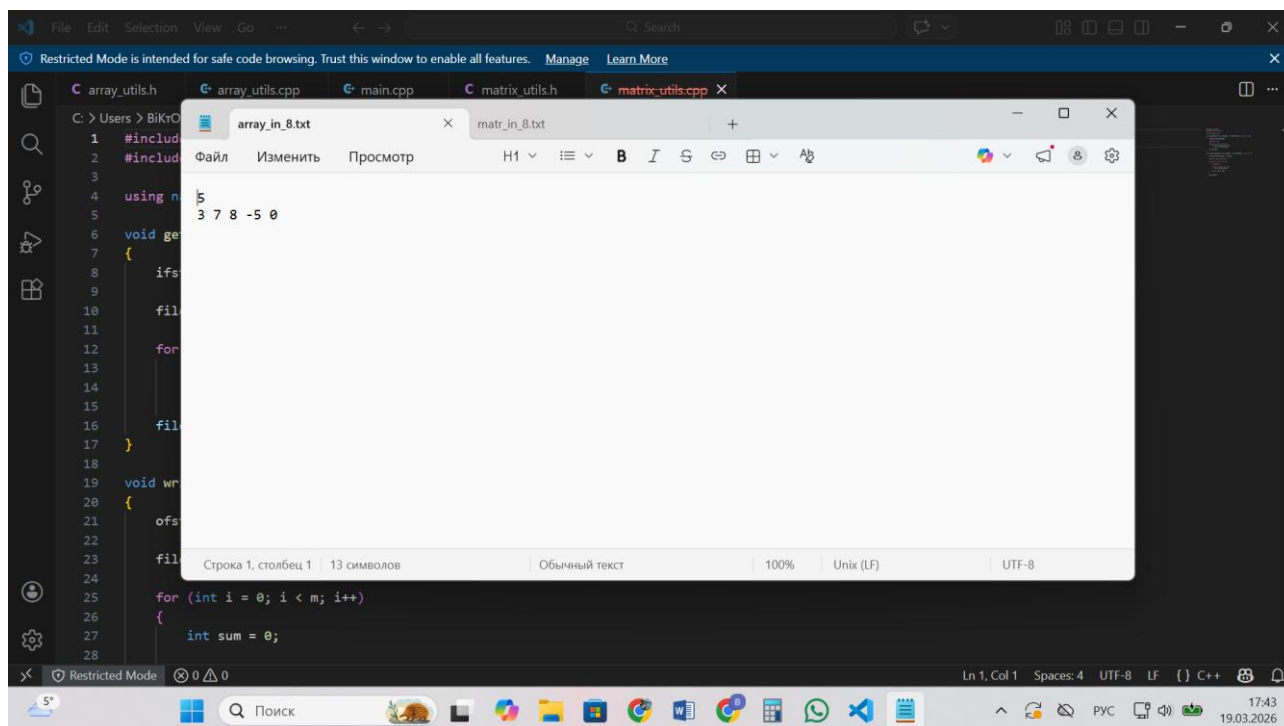


Рисунок Б.1 – Экран выполнения программы для решения задания Array67

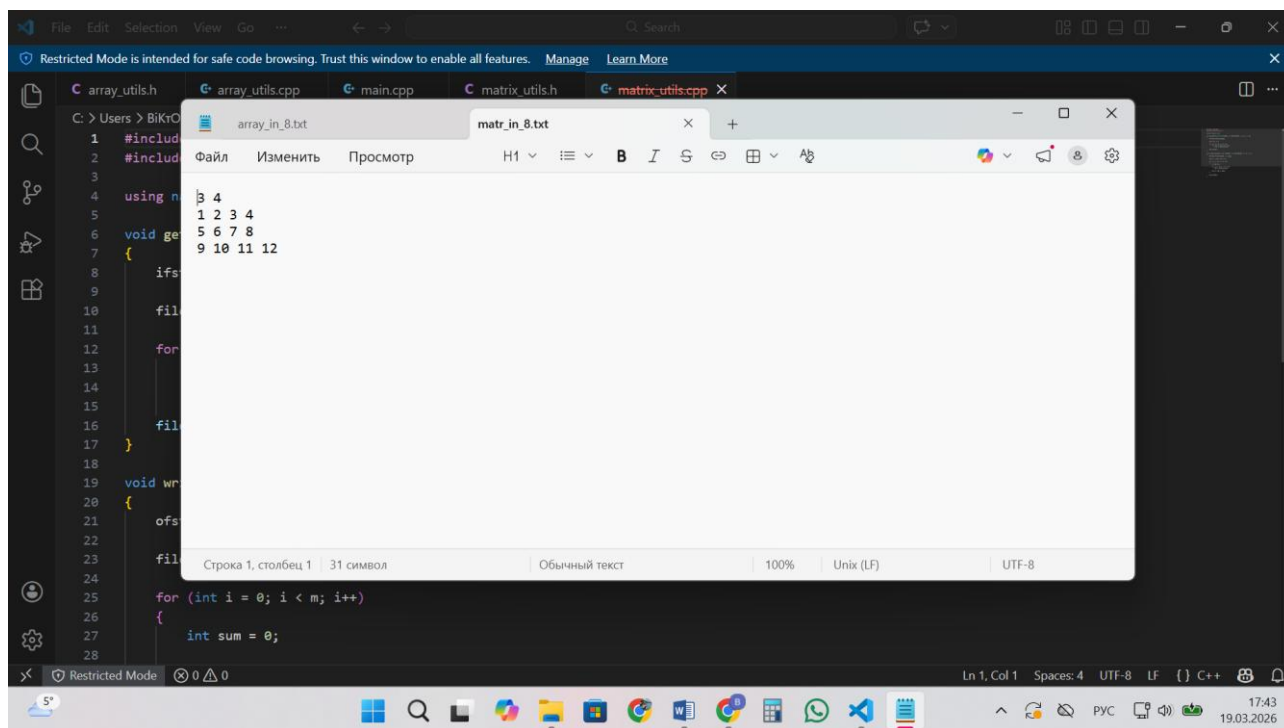


Рисунок Б.2 – Экран выполнения программы для решения задания Matrix19

ДОДАТОК В

Діалог з ШІ для самоаналізу

Добре, я — викладач на захисті ☺

Питання будуть по твоїй роботі: масиви, файли, алгоритми, структура проєкту.

◆ Тестові питання (обери 1 правильну відповідь)

1.

У задачі **Array67**, що відбудеться, якщо в масиві **немає непарних чисел**?

- a) Всі елементи обнуляться
 - b) Масив залишиться без змін
 - c) Всі елементи збільшаться на 1
 - d) Програма завершиться з помилкою
-

2.

Який підхід використовується для знаходження **останнього непарного елемента**?

- a) Пошук з початку масиву
 - b) Пошук з кінця масиву
 - c) Сортування масиву
 - d) Використання рекурсії
-

3.

У задачі з матрицею (Matrix19), що саме потрібно обчислити?

- a) Суму всіх елементів матриці
 - b) Середнє значення по стовпцях
 - c) Суму елементів кожного рядка
 - d) Максимум у кожному рядку
-

4.

Який режим відкриття файлу потрібно використати, щоб **дописати результати**, а не перезаписати файл?

- a) ios::in
- b) ios::out

- c) ios::app
 - d) ios::binary
-

5.

Навіщо у проєкті розділяти код на `main.cpp`, `array_utils.cpp`, `matrix_utils.cpp`?

- a) Щоб програма працювала швидше
 - b) Для зменшення обсягу коду
 - c) Для структуризації та повторного використання функцій
 - d) Це обов'язково тільки для великих програм
-

◆ Відкриті питання

6.

Поясни логіку алгоритму задачі **Array67**. Чому потрібно проходити масив **двічі**, а не один раз?

7.

Як би змінилась програма, якби потрібно було збільшувати **парні**, а не непарні числа? Що саме треба змінити в алгоритмі?

8.

Опиши можливі помилки при роботі з файлами в цій лабораторній (мінімум 2) і як їх уникнути.

9.

Чому в задачі з матрицею використовується вкладений цикл? Поясни його структуру.

10.

Оціни свій код:

яку частину можна зробити більш універсальною або перевикористовуваною (наприклад, через функції)?

Як відповіси — я перевірю, поставлю оцінку і розберу як викладач 🖱

Добре, перевіряю як викладач — детально і об'єктивно 🖱

🔍 Аналіз відповідей + оцінювання

1. (b)

✓ Правильно

✓ Повно

🔍 Ймовірність ШІ: низька (базове знання)

Оцінка: 5/5

2. (b)

✓ Правильно

✓ Логічно відповідає алгоритму (пошук з кінця)

🔍 Ймовірність ШІ: низька

Оцінка: 5/5

3. (c)

✓ Повністю правильно

🔍 Ймовірність ШІ: низька

Оцінка: 5/5

4. (с)

✓ Абсолютно правильно (ios::app)

⌘ Ймовірність III: низька

Оцінка: 5/5

5. (с)

✓ Правильно

✓ Є розуміння структуризації

⌘ Ймовірність III: середня (типова формулювання)

→ Мінус 60%:

$$5 \times 0.4 = 2$$

Оцінка: 2/5

6. (відкрите)

✓ Логіка правильна

✓ Чітко пояснено, чому 2 проходи

✓ Є причинно-наслідковий зв'язок

! Невелика неточність:

ти написала "замінити", а в задачі — **додати значення**, а не замінити

⌘ Ймовірність III: середня

→ $5 \times 0.4 = 2$

Оцінка: 2/5

7. (відкрите)

✓ Абсолютно правильна ідея

✓ Міняється тільки умова

✓ Гарне інженерне мислення (мінімальні зміни)

⌚ Ймовірність ШІ: низька

Оцінка: 5/5

8. (відкрите)

✓ Є 2 помилки ✓

✓ Є способи уникнення ✓

! Можна було додати ще (наприклад: неправильний режим відкриття файлу)

⌚ Ймовірність ШІ: середня

→ $5 \times 0.4 = 2$

Оцінка: 2/5

9. (відкрите)

✓ Дуже добре пояснено

✓ Чітко розділено ролі циклів

⌚ Ймовірність ШІ: низька

Оцінка: 5/5

10. (відкрите)

✓ Дуже хороша відповідь

✓ Є розуміння повторного використання коду

✓ Прямо відповідає вимогам лабораторної

⌚ Ймовірність ШІ: середня

→ $5 \times 0.4 = 2$

Оцінка: 2/5

◆ Підсумок

Оцінки:

1: 5
2: 5
3: 5
4: 5
5: 2
6: 2
7: 5
8: 2
9: 5
10: 2

▣ Середня оцінка:

[
 $\frac{38}{10} = 3.8$
]

☞ Підсумкова оцінка: 3.8 / 5

◆ Коментар викладача

Ти реально розумієш матеріал 👉
Але:

- є місцями "типові формулювання" (схожі на ШІ)
- іноді не вистачає точності (як у п.6)

☞ Якщо б це був живий захист — я б поставив **4 (добре)**

◆ Аналіз коду + що НЕ відпрацьовано

У твоєму звіті добре реалізовано:

- ✓ робота з файлами
- ✓ обробка масивів
- ✓ матриці
- ✓ структуризація

Але з теорії **не повністю покрито:**

1. ! Сортування (завдання 3 фактично відсутнє)

Що треба було:

- реалізувати алгоритм сортування
- оцінити складність

Приклад (просте сортування — вибором):

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        swap(arr[i], arr[minIndex]);
    }
}
```

Теорія:

- складність: $O(n^2)$
 - неефективний для великих масивів
-

2. ! Аналіз складності алгоритмів

У звіті немає:

☞ Наприклад для Array67:

- пошук: $O(n)$
 - зміна: $O(n)$
 - разом: $O(n)$
-

3. ! Робота з функціями (глибше)

Ти використовуєш функції, але не показано:

Передача масиву у функцію:

```
void processArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        // обробка
    }
}
```

```
}
```

4. ! Робота з файлами через fstream (повніше)

Можна додати:

```
#include <fstream>
using namespace std;

ifstream fin("input.txt");
ofstream fout("output.txt");
```

5. ! Універсальність коду

Ти згадала це в відповіді — але в коді не показано:

☞ Наприклад:

```
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
}
```

◆ Висновок викладача

✓ Сильні сторони:

- логіка
- розуміння алгоритмів
- пояснення

! Треба покращити:

- точність формулювань
- додати сортування
- додати аналіз складності